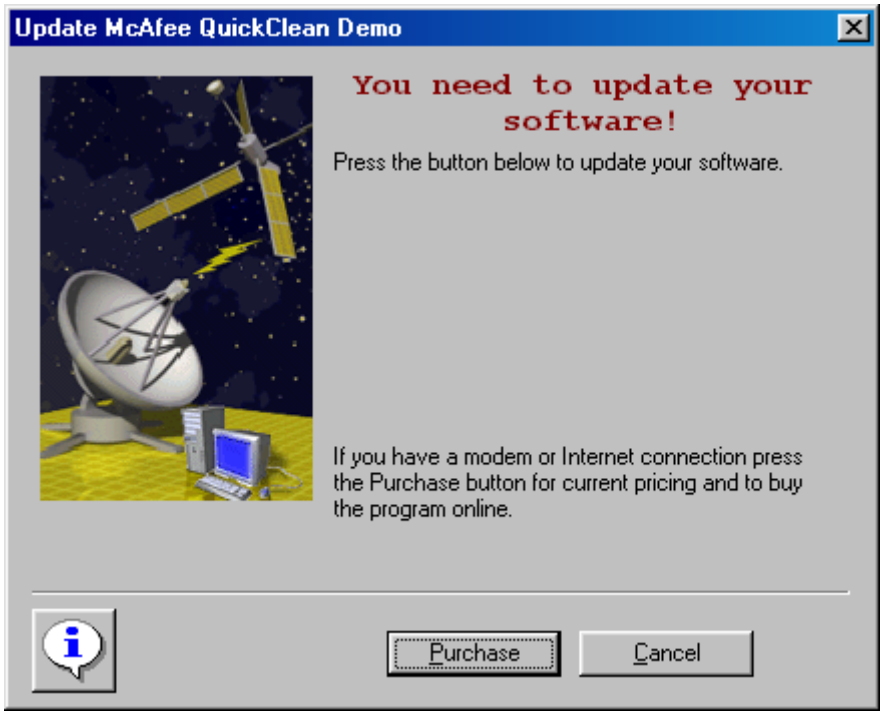
Programa:	McAfee Quick Clean 1.01.0084.0	
Protección:	Trial 30 días	
Objetivos:	Eliminar la protección temporal	
URL descarga:	http://software.mcafee.com	
Dificultad:	Sabiendo lo que es la IAT no problema… ;DDD	
Herramientas:	SoftIce + UltraEdit32.	
Cracker:	SyXe'05 [hAcK-c0d3d]	Fecha: 12-05-2005

Preliminares...

Estamos ante otro producto de la enorme compañía Network Associates technologies, (parece ser la Network Trademarks de toda la vida), éste implementa un TimeTrial de 1 mes, pasado este tiempo todas aquellas funciones a las que nos habremos acostumbrado dejarán de funcionar quedando totalmente inútil el programa, toda una mierda como podréis ver, está claro que como estrategia comercial no está del todo mal pero a nosotros no nos va muy bien que digamos, trataremos de arregalrlo **;D**

Comencemos...

Lo primero que haremos una vez presentada ya la víctima es forzar un primer ataque, adelantemos el reloj del windowx para que el programa expire, de esta manera cuando queramos ejecutarlo nos mostrará este mensaje:



Está claro que no podemos evaluar más el programa (o al menos eso creen quienes lo programaron), pues si de verdad pensaban que esto nos va a detener en nuestros objetivos van mal encaminados pues todo lo contrario, sí así es porque utilizaremos el mensaje como un punto de entrada al código del programa, si le seguimos la pista al cuadro de diálogo hacia atrás (con ***F12*** en el ***SoftIce***) llegaremos hasta el call responsable de todo, de esta manera sólo tendremos que estudiar el código que por allí haya y modificarlo de manera que se comporte como nosotros necesitamos.

Sin más dilación cargamos el archivo ***Uni.exe*** en el ***Symbol Loader*** ya que este es el programa principal, ¿qué como lo sé? Pues porque o he comprobado, antes de adelantar el reloj eché un vistazo a todos los archivos que trae que no son pocos, así me cercioro de lo que es cada uno de ellos, así después me será más fácil el trabajo. Charlas aparte, lo lanzamos desde el loader del ***SoftIce*** (Sice de ahora en adelante) pulsando sobre el botón de las ruedecillas dentadas para que rompa el Sice, en ese momento estaremos justo en la primera instrucción del archivo ejecutable que estamos traceando (esto siempre será así), como comentario decir que si el ejecutable está ***empakado*** (con UPX, tElock, ASPack, etc.,) el Sice no saltará, para que lo haga con ProcDump u otro similar le deberemos cambiar las propiedades de las secciones a ***E0000020*** pues muy posiblemente estén en ***C0000040*** u otra parecida, dicho esto seguimos pues no es el caso que nos atañe puesto que este no está comprimido, lanzado nuestro programa si vamos

traceando con ***F10*** iremos llegando a los sucesivos calls que pinten la nag de expired, cada vez que esto ocurra deberemos poner un bpx en la dirección del call y reiniciar el proceso saliendo del ***Sice***, cerrando el programa y volviéndolo a lanzar desde el ***Loader***, esto lo haremos hasta que veamos que la nag empieza a pintarse por partes: primero el fondo y luego un bucle en que termine de pintarla, en este momento pulsamos ***F12***, saldrá en pantalla aceptamos para retornar al Sice justo debajo del call responsable de todo esto, en muchos casos deberemos pulsar otro ***F12*** ya que de modificar este call luego la ventanita About no se vea puesto que se pinta también aquí, en este caso deberemos llegar a:

```
:1000604D E89EE8FFFF        call 100048F0    ;Interesante llamada!
:10006052 85C0              test eax, eax    ;Verifica el retorno -> ¿es eax=0?
:10006054 750F              jne 10006065     ;si es eax diferente de cero saltará y veremos la nagscreen!
:10006056 8BCE              mov ecx, esi
:10006058 E803F2FFFF        call 10005260    ;Interesante llamada!
:1000605D 85C0              test eax, eax    ;Verifica el retorno -> ¿es eax=0?
:1000605F 7404              je 10006065      ;si es eax diferente de cero saltará y veremos la nagscreen!
:10006061 33DB              xor ebx, ebx
:10006063 EB19              jmp 1000607E     ;si llegamos aquí no veremos la nag

* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
|:10006054(C), :1000605F(C)
|
:10006065 8BCE              mov ecx, esi
:10006067 E8E4F5FFFF        call 10005650    ;Call_Nag…!!!
:1000606C 8BD8              mov ebx, eax
:1000606E EB0E              jmp 1000607E
```

La zona muestra el lugar donde el programa saca la ***nag*** de ***expired***, tenemos dos saltos que nos meten de pleno en la llamada, están en ***:10006054*** y ***:1000605F***, cualquiera de ellos que se ejecute hará que veamos el cuadro de diálogo que precisamente queremos evitar. Examinemos el ***call 10048f0*** que es quien controla el primer salto, entramos dentro del mismo para ver lo siguiente:

```
* Referenced by a CALL at Addresses:
|:10003259   , :1000604D
|
:100048F0 83EC10            sub esp, 00000010
:100048F3 56                push esi
:100048F4 8BF1              mov esi, ecx
:100048F6 83BEF001000001    cmp dword ptr [esi+000001F0], 00000001
:100048FD 7440              je 1000493F          ;salto_BUENO!
:100048FF 83BEEC01000001    cmp dword ptr [esi+000001EC], 00000001
:10004906 7437              je 1000493F          ;salto_BUENO!
:10004908 8D442404          lea eax, dword ptr [esp+04]
:1000490C 50                push eax

* Reference To: KERNEL32.GetLocalTime, Ord:011Bh      ;Interesante llamada a GetLocalTime…!!!
                |
:1000490D FF1538A10110      Call dword ptr [1001A138]
:10004913 8D8E74010000      lea ecx, dword ptr [esi+00000174]
:10004919 8D542404          lea edx, dword ptr [esp+04]
:1000491D 51                push ecx
:1000491E 52                push edx
:1000491F E82C000000        call 10004950        ;Llamada determinante…!!!
:10004924 8BB684010000      mov esi, dword ptr [esi+00000184]
:1000492A 83C408            add esp, 00000008
:1000492D 85F6              test esi, esi
:1000492F 7404              je 10004935          ;salto_MALO!
:10004931 3BC6              cmp eax, esi
:10004933 720A              jb 1000493F          ;salto_BUENO!

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:1000492F(C)
|
:10004935 B801000000        mov eax, 00000001    ;flag de error activado...!!!
:1000493A 5E                pop esi
:1000493B 83C410            add esp, 00000010
:1000493E C3                ret


* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
```

```
|:100048FD(C), :10004906(C), :10004933(C)
:1000493F 33C0          xor eax, eax          ;flag de éxito activado… !!!
:10004941 5E            pop esi
:10004942 83C410        add esp, 00000010
:10004945 C3            ret
```

Podríamos investigar el valor de los flags ***[esi+000001f0]*** y ***[esi+000001ec]*** pero ciertamente he hecho el crack sin tenerlos en consideración así que explicaré otro caso aunque sería bueno recalcar que es posible que se pueda parchear de otra manera donde seguro uno o ambos flags tengan mucho que ver, si miramos la rutina vemos una llamada a la función ***GetLocalTime*** que lo que hace es obtener la fecha actual y ponerla en los registros, poco más abajo tenemos otra llamada esta vez es un ***call 010004950*** que resultará determinante para nosotros, veamos que podemos hacer con él...

Al Atake...

Primeramente lo que haremos será estudiarlo así por encima, veamos:

```
* Referenced by a CALL at Addresses:
|:1000491F   , :100049C8   , :10004A20   , :10004B3E   , :10004EFD
|:10005731   , :1000575B   , :1000584B   , :100081F4
:10004950 8B4C2404      mov ecx, dword ptr [esp+04]
:10004954 8B542408      mov edx, dword ptr [esp+08]
:10004958 56            push esi
:10004959 33C0          xor eax, eax
:1000495B 668B01        mov ax, word ptr [ecx]
:1000495E 33F6          xor esi, esi
   • • •                • • •
:10004981 33D2          xor edx, edx
:10004983 668B5106      mov dx, word ptr [ecx+06]
:10004987 8D0440        lea eax, dword ptr [eax+2*eax]
:1000498A 8D0480        lea eax, dword ptr [eax+4*eax]
:1000498D D1E0          shl eax, 1
:1000498F 2BC6          sub eax, esi
:10004991 5E            pop esi
:10004992 03C2          add eax, edx     ;finalmente el retorno en eax tras todas las operaciones…!!!
:10004994 C3            ret
```

He acortado un poco la rutina por motivos de tamaño ya que tampoco voy a escribir el programa entero aquí como es comprensible supongo, lo importante aquí es saber que el call retorna un valor en eax que no es más que el día actual del Trial en que nos encontramos, así si estamos en el primer día eax valdrá 1, si estamos en el tercero pues 3 y así sucesivamente, es de mencionar que dicho call es llamado varias veces lo que denota su importancia dentro del programa, ahora lo que haremos será obligar de alguna manera a la rutina a que retorne un valor que esté dentro de los 30 días Trial que nos da el programa como evaluación, de esta manera creerá que siempre estamos dentro de los límites de Trial permitidos ¿buena idea no?

Para arreglar esto simplemente debemos cambiar la instrucción:

```
:10004992 03C2          add eax, edx     POR:
:10004992 B001          mov al, 01
```

Aplicamos el parche con cualquier editor hexadecimal tipo ***UltraEdit32***, HexWorkShop u otros, salvamos los cambios con cualquier otro nombre y comprobamos a ver qué...

¿Qué...? vemos la nag todavía... aceptamos y entramos en el programa, bueno recapitulemos, tenemos un TimeTrial que ha caído descaradamente pero seguimos viendo la ventanita de expired, si traceamos el código en busca del lugar exacto donde esto ocurre nos encontramos con esto:

```
* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
|:10006054(C), :1000605F(C)
:10006065 8BCE          mov ecx, esi
:10006067 E8E4F5FFFF    call 10005650    ;Call_Nag...!!!
:1000606C 8BD8          mov ebx, eax
```

Aquí hemos llegado, el mensaje lo saca en ***:10006067***, si miramos estamos en un lugar que viene referenciado desde dos puntos distintos del código que son :10006054 y ***:1000605F***, vayamos allí:

```
:1000604D E89EE8FFFF        call 100048F0
:10006052 85C0              test eax, eax
:10006054 750F              jne 10006065     ;Salto_MALO...!!!
:10006056 8BCE              mov ecx, esi
:10006058 E803F2FFFF        call 10005260
:1000605D 85C0              test eax, eax
:1000605F 7404              je 10006065      ;Salto_MALO...!!!
```

que está justo encima del call, entre medio de estos dos saltos y el call hay un jmp que de no producirse ninguno de estos dos saltos se ejecutará saltando por encima del call, con esto la solución está clara: nopear ambos saltos. Sustituimos entonces:

```
:10006054 750F      jne 10006065     POR:
:10006054 9090      nop (2 veces)

:1000605F 7404      je 10006065      POR:
:1000605F 9090      nop (2 veces)
```

Hechos estos cambios parece ser que el ***TimeTrial*** ha caído verdad, pues esto no es del todo cierto ya que nos surge un imprevisto de última hora, no sé como me he dado cuenta, ha sido por casualidad ya que no sé por qué motivo he ido a ejecutar el programa y cual ha sido mi sorpresa al ver que no se ejecuta, no responde a un doble clic del ratón parece que se va a ejecutar pero es como si se apagara y sin dar mensaje de error alguno...!!! esto es rarísimo, pero si habíamos pillado las llamadas correctas a ***Ecapi.dll*** y que hemos falseado de manera que le lleguen valores válidos al programa conforme estamos en el primer día del periodo de evaluación!! Está claro que está poniéndonos las cosas difíciles ;(

Explicaré el método que utilicé para reparar esto: Lo primero que hice fue cargar en el Loader cualquier aplicación asociada a la aplicación: Quick Clean, Quick Clean Lite, Quick Clean Installation Monitor, etc., esta última (Quick Clean Installation Monitor) es la que he utilizado para resolver el tema, todo pasa por lanzar la aplicación desde el Loader e ir traceando con ***F10*** hasta encontrar algo que no nos convenza, tenemos la gran suerte de saber dónde saca el programa así que por algún motivo no llegamos a ese call, cuando localicemos el ***"error"*** lo solventamos y tema resuelto, esto es cuestión de paciencia (yo no tengo mucha) y tranquilidad, doy por hecho que cada uno de vosotros no tendréis problema en tracear el código y como no tiene mayor interés que el de resolver el nuestro problemilla daré directamente el lugar donde el programa tomaba unos valores que no nos iban bien, decir además que el tema venía de otra de sus innumerables librerías, ***EComWr.dll*** en este caso, resulta que las aplicaciones del pack van llamando a esta dll y esperan un valor que es el retorno, todo esto a través de un call que devolverá en eax un resultado de si todo ha ido bien o si en cambio ha ido mal (respecto a nosotros claro), esto ocurre porque el programa quiere saber si estamos en periodo de prueba todavía o si por el contrario ha expirado, lanza una llamada a una de sus librerías que le informará de esto, el resultado irá en un registro que será eax en este caso, verificará dicho valor y actuará en consecuencia, el tema está claro ¿no? Por eso decía de localizar el call responsable puesto que siempre suelen ser llamadas de esta manera se comunica el programa con sus librerías, etc., si hemos seguido el método de traceado desde el inicio con el Sice habréis llegado todos a esta zona:

```
:10001525 7514              jne 1000153B
:10001527 837C240401        cmp dword ptr [esp+04], 00000001
:1000152C 7508              jne 10001536
:1000152E 8B442420          mov eax, dword ptr [esp+20]
:10001532 85C0              test eax, eax
:10001534 7505              jne 1000153B

* Referenced by a (U)nconditional or (C)onditional Jump at Address:
|:1000152C(C)
|
* Possible Reference to String Resource ID=00001: "Unable to load ECAPI.DLL."
                |
:10001536 BE01000000        mov esi, 00000001    ;flag de éxito activado => esto es bueno para nosotros ;DDD

* Referenced by a (U)nconditional or (C)onditional Jump at Addresses:
|:100014E1(C), :10001525(C), :10001534(C)
|
:1000153B 8BC6              mov eax, esi         ;eax toma el valor final de esi…!!!
:1000153D 5E                pop esi
:1000153E 83C410            add esp, 00000010
:10001541 C3                ret                  ;Y retornamos...
```

En este trozo de código se decide si estamos o no en modo Trial o hemos expirado el plazo de prueba, eso lo deducimos porque si retornamos con eax=1 entramos en el programa (Trial) y si lo hacemos con ***eax=0*** no entramos (expired), si queremos salir con ***eax=1*** deberemos:

- O bien nopear los 3 saltos que van a :1000153b o bien cambiar:

```
:1000153B 8BC6      mov eax, esi      POR:
:1000153B B001      mov al, 01
```

Cada uno parchee como desee y le vaya mejor, es cuestión de gustos, lo importante es que saliendo de aquí con eax=1 llegaremos al programa que es nuestro retorno válido **;-)**

Y ahora sí caen las limitaciones del programa, no vemos más nagscreen ni nada de nada, ahora sí tenemos el crack del progama ya consumado **;DDD**

Notas Finales...

Y con esto damos por finalizado el tuto, espero que os haya gustado con esa intención lo he hecho, como siempre decir que está orientado hacia aquellas personas que empiezan en esto y quieren aprender, por ellos van estos tutos que hago y seguiré haciendo, pues cuando yo comenzaba me gustaba leer tutoriales que otras personas escribían, en mi caso leía mucho los de ESTADO+PORCINO que escribía unos muy buenos tutos, últimamente sigo leyendo de Ricardo Narvaja, +NCR, Black Fenix, AuspeX, DGRojo, R!SC, +Tsehp, etc., pues son innumerables. Ánimo a tod@s y a crackear que son dos días...jeje

Y fin con esto damos por finalizado el tuto, espero que os haya gustado, pronto volveremos con otros, si alguien quiere contactar conmigo por cualquier tema, ya sea una duda, una sugerencia o como no una crítica siéntase libre de escribirme a cualquiera de estas direcciones:

syxe00@yahoo.es
syxe00@hotmail.com